

Distributed Systems

(4th edition, version 01)

<https://www.distributed-systems.net/>

Chapter 02: Architecture

Last updated: September 9, 2026

Objective:

We look at three examples of distributed system architectures.
Each is very powerful and common.

- 1 | Three-tier architecture
- 2 | Client-server architecture
- 3 | Cloud computing

I. Three-Tier Architecture Diagram

I. Three-Tier Architecture Diagram

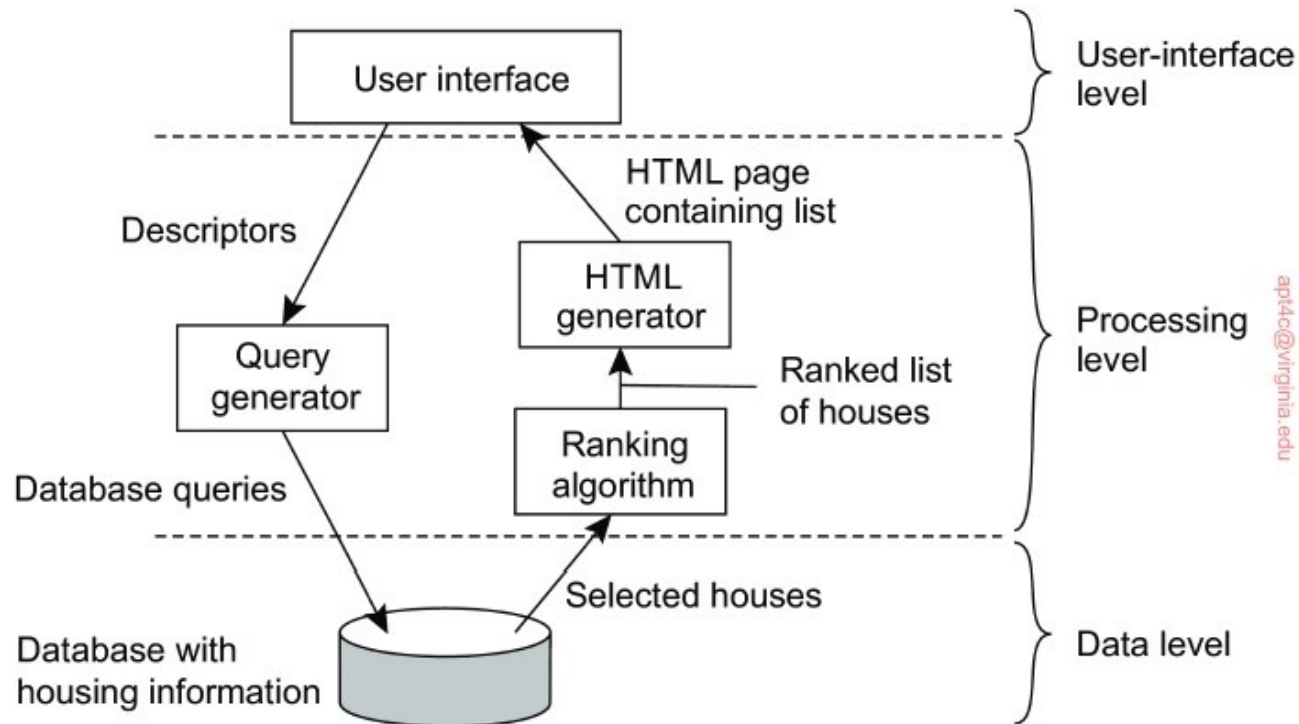


Figure 2.4: The simplified organization of an Internet search engine for housing, into three different layers.

II. Simple Client-Server Architecture

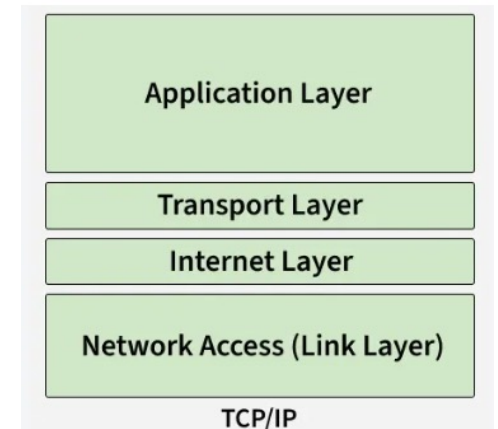
II. Background: Connection-Oriented Protocol

Most common is **TCP/IP** = Transmission Control Protocol / Internet Protocol

Standard “language” of the internet and most modern networks

Defines how devices connect, send, and receive data reliably

Nearly all internet application protocols use TCP/IP connections

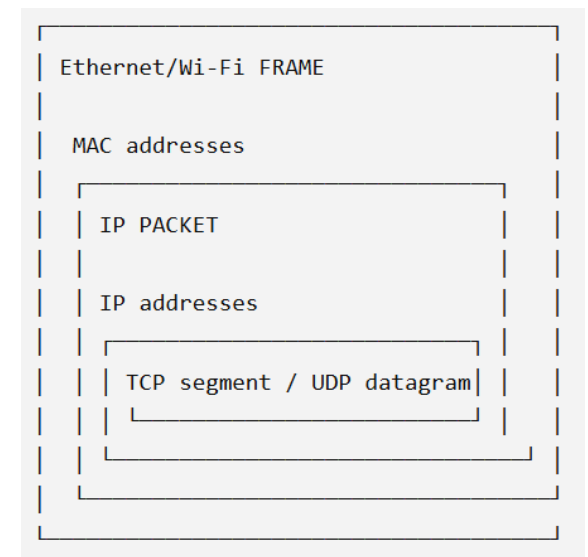


II. Background: Connection-Oriented Protocol

A **frame** is unit of data used to move information across local network

Typically consists of:

- > Source MAC address - who sent the data
- > Destination MAC address - who should receive it on local network
- > Payload - usually an IP **packet**
- > Error-checking information



II. Background: Connection-Oriented Protocol

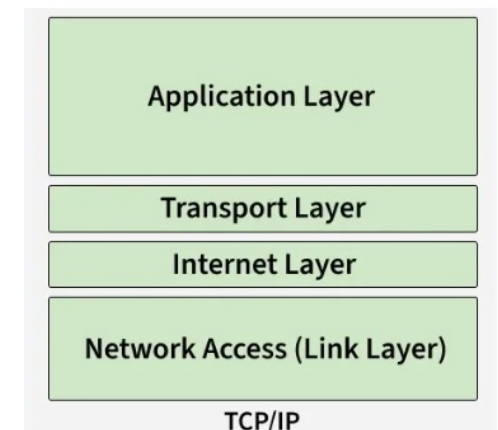
Function of each Layer

Application: Provides network services to applications (HTTP, DNS, email, etc.).

Transport: Provides end-to-end communication between applications; handles ports, reliability ordering, and flow control (TCP/UDP).

Internet: Handles IP addressing and routing packets between networks.

Network Access: Moves frames across the local network/link using technologies such as Ethernet and Wi-Fi.



II. Connection-Oriented Protocol

Benefits of TCP/IP:

Reliability

- TCP breaks data into packets, numbers them, and ensures all arrive in order
- Lost or corrupted packets get retransmitted automatically.

II. Connection-Oriented Protocol

Benefits of TCP/IP:

Reliability

- TCP breaks data into packets, numbers them, and ensures all arrive in order
- Lost or corrupted packets get retransmitted automatically.

Interoperability

- Every major OS, router, and application supports TCP/IP.

II. Connection-Oriented Protocol

Benefits of TCP/IP:

Reliability

- TCP breaks data into packets, numbers them, and ensures all arrive in order
- Lost or corrupted packets get retransmitted automatically.

Interoperability

- Every major OS, router, and application supports TCP/IP.

Scalability

- Scales to billions of devices

II. Connection-Oriented Protocol

Benefits of TCP/IP:

Reliability

- TCP breaks data into packets, numbers them, and ensures all arrive in order
- Lost or corrupted packets get retransmitted automatically.

Interoperability

- Every major OS, router, and application supports TCP/IP.

Scalability

- Scales to billions of devices

Flexibility

- Supports multiple application protocols: HTTP(S), FTP, SSH, email, streaming

II. Connection-Oriented Protocol, contd.

Mechanics of TCP/IP:

Before client requests service, connection set up w/server

Server uses same connection to send reply message

When communication finished, connection torn down

II. Connection-Oriented Protocol, contd.

What happens if we don't use a connection?

II. Simple Client-Server Architecture (No connection)

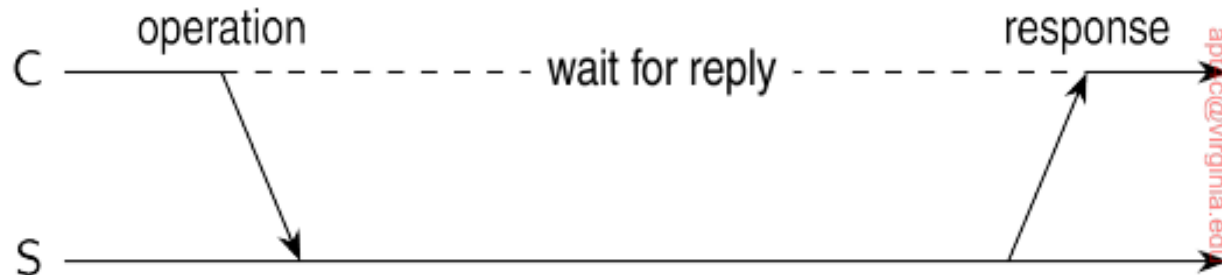


Figure 2.16: General interaction between a client C and a server S. C sends the operation oper and waits for the response from S.

- 1) Client packages message for server, identifying the service it wants, along with necessary input data
- 2) Message sent to server
- 3) Server will wait for incoming request, process it, and package results in a reply message sent to client

II. Simple Client-Server Architecture (No connection), contd.

When it works, this is efficient ...

... but what if the result doesn't come back?

II. Simple Client-Server Architecture (No connection), contd.

When it works, this is efficient ...

... but what if the result doesn't come back?

- 1) Was original request lost?
- 2) Did transmission of response fail?

We won't know

II. Simple Client-Server Architecture (No connection), contd.

When it works, this is efficient ...

... but what if the result doesn't come back?

- 1) Was original request lost?
- 2) Did transmission of response fail?

We won't know

In first case, might resend. In second case, don't want to resend.

Transaction that can safely be repeated is called *idempotent*

II. Simple Client-Server Architecture (No connection), contd.

Examples:

Request 1: transfer \$10K from bank account

Not safe to resend

Request 2: Tell me bank account balance

Safe to resend

III. Cloud Computing

III. Cloud computing

The model for cloud computing can be regarded as four layers

III. Cloud computing

The model for cloud computing can be regarded as four layers

Hardware: Processors, routers, power and cooling systems. Customers normally never get to see these.

III. Cloud computing

The model for cloud computing can be regarded as four layers

Infrastructure: Deploys virtualization techniques. Evolves around allocating and managing virtual storage devices and virtual servers.

Hardware: Processors, routers, power and cooling systems. Customers normally never get to see these.

III. Cloud computing

The model for cloud computing can be regarded as four layers

Platform: Provides higher-level abstractions for storage and such. Example: Amazon S3 storage system offers an API for (locally created) files to be organized and stored in so-called **buckets**.

Infrastructure: Deploys virtualization techniques. Evolves around allocating and managing virtual storage devices and virtual servers.

Hardware: Processors, routers, power and cooling systems. Customers normally never get to see these.

III. Cloud computing

The model for cloud computing can be regarded as four layers

Application: Actual applications, such as office suites (text processors, spreadsheet applications, GenAI services). Comparable to the suite of apps shipped with OSes.

Platform: Provides higher-level abstractions for storage and such. Example: **Amazon S3** storage system offers an API for (locally created) files to be organized and stored in so-called **buckets**.

Infrastructure: Deploys virtualization techniques. Evolves around allocating and managing virtual storage devices and virtual servers.

Hardware: Processors, routers, power and cooling systems. Customers normally never get to see these.

III. Cloud computing

